METADATA AUTOMATION: A TF-IDF AND NEAREST NEIGHBORS APPROACH

Emily Hartley†, Roopal Bhatnagar†, and Kurt Michels†
† Critical Path Institute, Tucson, AZ, USA

CRITICAL PATH INSTITUTE 15 YEARS

DATA COLLABORATION® CENTER

The goal of this challenge was to develop a tool to automate the annotation of metadata fields and values in structured biomedical data files with the best candidate Common Data Element (CDE) matches from the Cancer Data Standards Registry and Repository (caDSR).

C-Path's Approach:

We approached this challenge with the idea that it was essentially a fuzzy matching problem. We implemented our model in Python 3.6 utilizing several Python libraries including Scikit-learn, FuzzyWuzzy, and pandas. Our model consisted of the following steps: data processing and representation, searching for the nearest neighbors, and final selection and ranking of the matches.

Datatype Inference:

We inferred 2 pieces of information for each column in the input data file (the query).

- **Datatype:** For each observed value, we checked if it was one or more of 4 specified datatypes (string, datetime, number, Boolean) and created a list of all the unique possibilities.
- **Enumeration Status:** We based our inference on the number of distinct values (n_d) and the total number of rows (N) in the column.
  - Non-Enumerated: $n_d \geq 30$ or $n_d > \frac{1}{2}N$
  - Enumerated: $0 < n_d < 30$ or $n_d \leq \frac{1}{2}N$
  - Unknown: $n_d = 0$

Term Frequency – Inverse Document Frequency (TF-IDF)

$TF(t,d) \times IDF(t)$: Measures how relevant a term t is to a document d in a set of documents.

$$TF(t,d) = \frac{\#\ of\ times\ t\ appears\ in\ d}{Total\ \#\ of\ terms\ in\ d}$$

$$IDF(t) = \log\left(\frac{Total\ \#\ of\ documents}{\#\ of\ documents\ containing\ t}\right)$$

For more information about C-Path's contributions to the Metadata Automation DREAM Challenge:

CONTACT: Amanda J. Borens, MS
Executive Director of Data Science, Critical Path Institute
aborens@c-path.org

Data Representation:

We utilized Scikit-learn's TfidfVectorizer class along with two custom n-gram analyzer functions to vectorize the data. The custom n-gram functions performed both text cleaning/pre-processing steps (e.g., adding or removing spaces, removing punctuation, transforming the string to lower or title case) along with splitting the string into chunks of length 2 (bigrams).

For the metadata, we fit the TfidfVectorizer separately to select caDSR fields (including twice to the CDE_LONG_NAME). To form the query, we fit the column header, observed values and inferred datatype to the corresponding caDSR TF-IDF vectors, applying the same analyzer function. Then we used Scikit-learn's ColumnTransformer class to concatenate the TF-IDF vectors into a single feature space.

Bigrams with Spaces

primary_diagnosis →
[' P', 'Pr', 'ri', 'im', 'ma', 'ar', 'ry', 'y ', ' D', 'Di', 'ia', 'ag', 'gn', 'no', 'os', 'si', 'is', 's ']

Primary Tumor Diagnosis Text →
[' P', 'Pr', 'ri', 'im', 'ma', 'ar', 'ry', 'y ', ' T', 'Tu', 'um', 'mo', 'or', 'r ', ' D', 'Di', 'ia', 'ag', 'gn', 'no', 'os', 'si', 'is', 's ', ' T', 'Te', 'ex', 'xt', 't ']

Bigrams without Spaces

survivaltimeindays →
['su', 'ur', 'rv', 'vi', 'iv', 'va', 'al', 'lt', 'ti', 'im', 'me', 'ei', 'in', 'nd', 'da', 'ay', 'ys']

Patient Survival Interval Day Count →
['pa', 'at', 'ti', 'ie', 'en', 'nt', 'ts', 'su', 'ur', 'rv', 'vi', 'iv', 'va', 'al', 'li', 'in', 'nt', 'te', 'er', 'rv', 'va', 'al', 'ld', 'da', 'ay', 'yc', 'co', 'ou', 'un', 'nt']

This table shows the fields from the caDSR and input files used to construct the TF-IDF matrix and vectors passed into the Nearest Neighbor function.

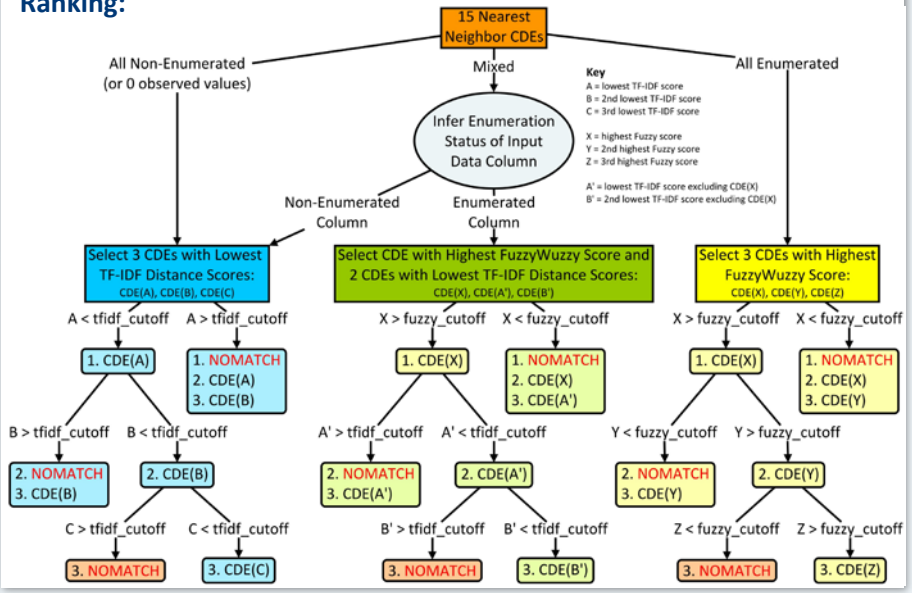
caDSR Field	Input File Field	Analyzer Function
CDE_LONG_NAME	Column Header	Bigrams – no spaces, lowercase
CDE_LONG_NAME	Column Header	Bigrams – with spaces, title case
QUESTION_TEXT	Column Header	Bigrams – with spaces, title case
DEFINITION	Column Header	Bigrams – with spaces, title case
CDE_SHORT_NAME	Column Header	Bigrams – no spaces, lowercase
DEC_LONG_NAME	Column Header	Bigrams – with spaces, title case
PERMISSIBLE_VALUES	List of unique observed values	Default word-grams
DATATYPE (condensed categories)	List of inferred datatypes for observed values	Default word-grams

Search:

The metadata TF-IDF matrix and query vectors were passed to Scikit-learn's Nearest Neighbor class which, for each query vector, returned the 15 nearest neighbor CDEs and their associated distance score.

Value Domain: If any of the 15 returned CDEs were enumerated, the FuzzyWuzzy library was utilized to perform an additional value domain search to help rank the matches. FuzzyWuzzy uses Levenshtein Distance to compare the observed values to the permissible values. The Levenshtein Distance scores for all the observed values were then averaged together to create a total score for the column.

Ranking:



End-to-End Learning:

We manually tested different parameter settings against the provided real data test files. In our testing we found that some parameter settings worked well for one subset of the test datasets and poorly for the others, while a different setting would work less well for a different (sometimes overlapping, sometimes distinct) subset of the datasets but also less poorly for the other datasets. Because of this, we focused on trying to maximize the average score across all the test datasets.

Acknowledgments: Funding - The Metadata Automation DREAM Challenge is fully supported by funds provided through the Cancer Moonshot℠. Work by Sage Bionetworks staff for the Challenge is supported by the NCI Cancer Systems Biology Consortium (CSBC) and Physical Sciences-Oncology Network (PS-ON).
The following contributors are acknowledged for data provided: The Applied Proteogenomics Organizational Learning and Outcomes (APOLLO) Network and The Cancer Imaging Archive (TCIA)